Destructible Structure Builder Manual Version: 1.0.0
Compatibility: Unity 6.3 LTS, 6.0 LTS, 2022.3 LTS

Destructible Structure Builder (DSB) is a Unity editor toolkit for authoring modular destructible structures. Assemble structures from connections, members, and surfaces directly in the Unity Editor, then let DSB handle runtime damage propagation, collapse behavior, and debris generation.

Table of Contents

1. Quick Start Tutorial

1.1 Required Assets & Managers (Project-Level Setup)

Step 1: Setup

1. Open a new scene.
2. Launch **Tools → Destructible Structure Builder**.
3. Ensure a **DSBGlobalSettings** asset is assigned (create one if prompted).
4. Create **EffectsLibrary** via **Assets → Create → DSB → Effects Library** or use the default one under **Samples/** and assign it in **DSB window → Settings → Effects Profiles**. (see Assigning Effects).

- **URP/HDRP users:**
  Sample materials target the Built-in RP.
  Convert them via **Edit → Render Pipeline → ... Upgrade Project Materials**.
- **Navigation Rebaking:**
  Install Unity's AI Navigation package (`com.unity.ai.navigation`) for navmesh rebaking support.

1.2 First Structure Walkthrough

- **Create Structure:** Place the root connection node via **Create Structure**. Once placed, the mode will automatically switch to **Grid Member Build**.
- **Members:** Add beams via **Grid Member Build** (snapped) or **Free Member Build** (any angle). Ghost previews show valid axes; click to confirm placement.
- **Surfaces:** With **Surface Build**, hover members to see ghost surfaces, drag for size, then click to finalize. Use **Apply Design** to bring in saved layouts.
- **Stairs:** Enter **Stair Build**. Placement is staged: click a member/connection face to set the stair anchor, click again to lock width, then click a final time to confirm the stair end point and spawn the full staircase.
- **Grounding:** Use **Grounded Toggle** to click connections and mark them **grounded**, or run **Auto-Detect Grounded** so connections that touch the ground are marked automatically. Grounded connections allow the stress simulation to propagate support through attached members.
- **Stress Basics:** Enable **Simulate Stress** to make supports, weight, and damage affect stability at runtime. Stress simulation works by propagating load from unsupported members toward grounded connections. If the load exceeds a member's structural strength, it fails and can trigger cascading collapse.
- Press **Play** and interact with the structure (collisions, projectiles, etc.) to see crumble, stress reactions, and debris pooling in action.
- **Stress Visualization:** Enable stress visualization while testing to see which parts are lightly loaded vs. close to failure. Make adjustments to **Density** (how heavy pieces behave) and **Structural Strength** (how much load members can tolerate) until collapse feels right for your scale.

Important: *All construction happens in the Unity Editor; the toolkit provides **no** in-game building or editing capabilities.*

1.3 Assigning Effects

- **EffectsLibrary** — per-event default effects (audio + particles). Create via **Assets → Create → DSB → Effects Library** and assign to **DSBGlobalSettings → Effects Profile** in the **DSB window → Settings → Effects Profiles** section.
- **MaterialEffectsLibrary** — per-material overrides. Create via **Assets → Create → DSB → Material Effects Library** and assign to **DSBGlobalSettings → Material Overrides Profile** in the **DSB window → Settings → Effects Profiles** section.
- A ready-to-use **EffectsLibrary** with preset audio and particle values lives under **Samples/**; assign it to start using included effects immediately.

1.4 Applying Damage to Structures

- **Use IDsbDamageable** – All runtime pieces (members, surfaces, connections) implement `IDsbDamageable`. Call `ApplyDamage(value)` from your own scripts when a raycast, projectile, or trigger hits a DSB object. Damage accumulates per voxel until the piece splits or crumbles.
- **Sample hit logic** – Raycast against the scene and call `ApplyDamage` on the collider you hit:

```
using Mayuns.DSB;

if (hit.collider.TryGetComponent<IDsbDamageable>(out var damageable))
    damageable.ApplyDamage(25f);
```

- **Collision Damage** – Optional impact damage can be driven by the **DSB window → Settings → Collision Damage** section. Configure **Collision Trigger Layers** (which incoming layers are allowed to trigger impact damage), **Collision Target Layers** (which overlap candidates can receive impact damage), **Collision Impulse Threshold**, **Collision Impulse Sensitivity**, **Collision Base Damage**, **Collision Base Radius**, and **Collision Damage Max Radius**. Structures and detached structural groups are both affected by collision damage.
- **Effects on crumble** – Ensure an **Effects Library** (and optional **Material Effects Library**) is assigned on the shared **DSBGlobalSettings** asset in **DSB window → Settings → Effects Profiles** so debris, particles, and audio play when damage destroys a voxel.

1.5 Core Workflow UX Explanations

- **Ghost Previews:** Hovering connections or members shows beams (members) or panels (surfaces) that update live with snapping options.
- **Rotation:** Rotate surface or stair placement direction with the overlay rotate button or the bound rotate hotkey.
- **Snapping:** Use the Scene overlay or hotkeys (*Shift*+*S* to toggle snap; arrow keys to lock X/Y/Z) to lock axes.

2. Core Editor Workflow

2.1 Build Modes

Build Overlay

Activating any build mode displays a small overlay in the Scene view. It offers **Rotate Design**, **Copy From Scene**, and **Cancel** buttons and shows the state of snapping and axis locks. Use the overlay or hotkeys (*Shift*+*S* to toggle snap; arrow keys to lock X/Y/Z) while placing members, surfaces, or stairs.

Hotkeys

Hotkeys are provided for exiting the build mode easily, toggling snapping, and rotating surfaces before placement. Toggle them via **DSB window → Hotkeys** and configure bindings from **Edit → Shortcuts** (search for *DSB/Structure Build*).

2.2 Surface & Member Authoring

This section covers the modes you use to build structural frames (connections + members) and surface elements (surfaces, stairs, and slopes). The practical order is:

1. Create or extend a frame with **Create Structure**, **Grid Member Build**, and/or **Free Member Build**.
2. Mark supports with **Grounded Toggle**.
3. Add surface geometry with **Surface Build**, **Stair Build**, or **Slope Build**.
4. Refine appearance and behavior with **Apply Design** and **Apply Material**.
5. Use **Delete** to remove placed pieces.

Member Authoring Workflow

Members are the backbone that all surfaces, stairs, and slopes must attach to. They are the load-bearing components of the structure, and they determine how stress propagates through the structure during simulation.

- **Create initial anchor**: Start with **Create Structure** and click in Scene view to place the first root connection.
- **Extend with Grid Member Build**: Use this for orthogonal framing and fast layout. Hover a valid start point (connection/member), verify the preview direction, then click to place the member and its end connection.
- **Extend with Free Member Build**: Use this for angled framing or non-grid geometry. Click a valid start point, move the cursor to define direction/length, then click to confirm the end.
- **Ground support intentionally**: In **Grounded Toggle**, click connections that should transfer support to the world. This is especially important when stress simulation is enabled.
- **Correct mistakes quickly**: In **Delete**, remove bad members/connections before adding surfaces. Deleting a connection also removes all connected members.

Important: *Members must have both a start and end connection before a new member can be drawn from them. Avoid deleting connections directly outside the dedicated delete tool to ensure members retain their required connections.*

Member authoring tips

- Use snapping + axis locks for cleaner member runs before adding surface content.
- You can also click on members and adjust their properties through the **Structural Member** inspector. Adjusting visual properties requires you to hit "Regenerate" to regenerate the geometry with the new settings.

Surface Build Workflow

Use **Surface Build** to place surface panels on member faces.

1. **Hover a valid host member** to display the ghost surface preview.
2. **Set surface width**:
    - If **Fixed Width** is disabled, click to start and drag along the host face to size width, then click again to lock width.
    - If **Fixed Width** is enabled, the tool uses the configured width and skips interactive sizing.
3. **Set surface height**:
    - If **Fixed Height** is disabled, move the cursor vertically and click to confirm height.
    - If **Fixed Height** is enabled, the configured height is used immediately.
4. **Finalize placement**. The surface is created using the active design source (active SurfaceDesign/default Surface Design), or an empty layout when no default design is assigned.

Surface placement controls

- **Rotate Design**: Use the Scene overlay button (or bound hotkey) before finalizing to rotate the design orientation in 90° steps.
- **Edge & Midpoint Snapping**: Pulls the surface toward nearby face edges and locks alignment to the member midpoint/endpoints along its length.

Stair Build Workflow

Use **Stair Build** when you need stepped geometry instead of a flat surface panel.

1. Click a valid member/connection face to anchor stair placement.
2. Move the cursor across the face to size first step width, then click to lock width.
3. Move to define stair run direction/length, then click to place.

Stairs follow the same surface-adjacent snapping concepts.

Slope Build Workflow

Use **Slope Build** to place angled surface-like panels (ramps/roof segments).

1. Hover a valid member or connection face to preview the slope.
2. **Width phase**:
    - If **Surface Width** is unlocked, click to set start, drag to size width, then click to lock.
    - If width is fixed, this phase is skipped.
3. **Length/direction phase**: move to pick end point (defines slope direction and run), then click to place.

Slope placement uses the active surface design source (active SurfaceDesign or default) and respects the surface snapping toggle. You can rotate design orientation before placement using the overlay rotate control/hotkey.

Surface Design Inspector

A SurfaceDesign asset defines the grid layout of cells used to generate surface geometry. The **Surface Design Inspector** appears when a `SurfaceDesign` asset is selected, and also when using the **Surface Build** or **Apply Design** modes. It provides a color-coded cell grid you can paint directly and then apply to scene surfaces.

Features

- **Interactive Grid** - Click/drag to edit cells live.
- **Brush Palette** - Select surface, window, triangle, sloped, or empty cell types.
- **Multi-cell Editing** - Paint across ranges by dragging.
- **Auto Grid Resize** - Row/column changes resize the backing cell list automatically.

How to Use

1. **Create/open a design** – **Assets → Create → DSB → Surface Design** or enter **Apply Design** and create/select one there.
2. **Select brush + orientation** – choose a cell type; for triangle/sloped cells, rotate orientation with **Rotate**.
3. **Paint the layout** – click/drag cells in the grid.
4. **Apply to scene surfaces** – use **Apply Design** mode and click target surfaces.

Apply Design Workflow

Use **Apply Design** to non-destructively change existing surfaces without having to delete and place the surface again.

- Enter **Apply Design** and ensure the intended SurfaceDesign is active.
- Click a surface in the scene to replace its cell layout with the active design.

Copy Surface Design From Scene

While in **Apply Design** or **Surface Build**, click **Copy Design From Surface in Scene** in the DSB window, then click an existing surface. Its live layout is copied into the active in-memory design/asset so you can reuse or save it.

Saving a Surface Design

1. Enter **Apply Design** and either create a new design or copy one from a scene surface.
2. Click **Save Design**.
3. Choose a name/location in the file picker to write a `.asset` containing rows, columns, cell types, and related material references.

Apply Material

- **Apply Material**: Click any connection/member/surface to assign the currently selected material.

Together with **Apply Design**, these are the refinement modes used after raw geometry placement. If desired, surfaces, connections, and members can all have their settings adjusted through their inspectors as well.

Texture Scaling

Every surface and structural member stores **Texture Scale X** and **Texture Scale Y**, representing meters per texture repeat along local axes. DSB divides the object's world size by these scales to derive tiling, then multiplies mesh UVs so textures repeat consistently regardless of dimensions. A scale of 1 means one meter per tile; 0.5 doubles tiling; 2 halves it; 0 falls back to one meter.

2.3 Structure Management

Structural Group Inspector

Selecting a **StructuralGroup component** exposes foldouts that mirror the custom inspector. The inspector also surfaces warnings for non-uniform scale, nested structures, missing grounded members, and over-capacity members.

- **HP**
  - Toggle **Invincible** to block damage.
  - *Surface Voxel Health*, *Window Voxel Health*, and *Member Voxel Health* control structure-wide health defaults.
- **Simulation**
  - **Density (kg/m³)** adjusts voxel mass (editable only outside Play Mode).
  - **Simulate Stress** enables the stress system for the structure.
  - **Structural Strength** controls the support capacity multiplier of each member (available when Simulate Stress is enabled).
  - **Show Stress Visualization** draws stress gizmos in the Scene view (available when Simulate Stress is enabled).
- **Utilities (Editor only)** – Disabled in Play Mode.
  - **Grounded Members → Auto-Detect Grounded** marks members touching grounded layers configured in **DSB window → Settings** (warns if no grounded layers are configured).
  - **Navigation → Add NavMesh Rebaker** attaches **StructureNavmeshRebaker** (disabled if already present).
  - **Voxel Operations → Regenerate Voxels** regenerates member, connection, and surface voxels (disabled while editing a prefab stage).
  - **Export → Export Combined OBJ** saves a single combined OBJ of all child meshes.
- **Component Count** – Shows counts for surfaces, members, connections, and total voxels.

2.4 Global Settings Reference

All project-level settings below live on the shared **DSBGlobalSettings** asset and are exposed in **DSB window → Settings**.

Layers

- **Grounded Layers** – Layers treated as world support for auto-ground detection.
- **Small Debris Layer** – Layer assigned to spawned small debris pieces.
- **Large Debris Layer** – Layer assigned to spawned large debris (detached proxy chunks).

Effects Profiles

- **Effects Profile** – Default destruction effects library (audio + particles).
- **Material Overrides Profile** – Per-material effect overrides.

Debris Behavior

- **Max Active Debris** – Runtime cap for active debris before oldest debris is culled.
- **Global Pool Size Limit** – Maximum pooled voxel/debris shell count before extra returned objects are destroyed.
- **Small Debris Explosiveness / Large Debris Explosiveness** – Random linear spawn impulse magnitude.
- **Small Debris Spin / Large Debris Spin** – Random angular spawn impulse magnitude.
- **Small Debris Lifetime / Large Debris Lifetime** – Time before debris returns to the pool.
- **Detached Group Lifetime** – Lifetime of detached structural groups before gradual crumble cleanup.

Collision Damage

- **Collision Damage Enabled** – Master toggle for collision-driven damage.
- **Collision Trigger Layers** – Incoming layers allowed to trigger impact damage.
- **Collision Target Layers** – Overlap-query target layers that may receive impact damage.
- **Collision Impulse Threshold** – Minimum impulse required to apply damage.
- **Collision Impulse Sensitivity** – Multiplier for impulse-based radius and damage growth.
- **Collision Base Damage** – Starting damage before scaling/falloff.
- **Collision Base Radius** – Starting radius before impulse scaling.
- **Collision Damage Max Radius** – Hard cap for scaled damage radius.

Debris Rendering

- **Small Debris Shadow Casting / Receive Shadows** – Default shadow behavior for small debris meshes.
- **Large Debris Shadow Casting / Receive Shadows** – Default shadow behavior for large debris meshes.

Collapse & Visibility

- **Large Collapse Mass Threshold** – Minimum detached mass required to raise large-collapse effects/events.
- **Hide Small Debris In Hierarchy** – Hides small debris objects in hierarchy.
- **Hide Large Debris In Hierarchy** – Hides large debris objects in hierarchy.
- **Hide Detached Groups In Hierarchy** – Hides detached structural groups in hierarchy.

Debug

- **Draw Collision Damage Overlap Gizmos** – Draws collision-damage overlap spheres.
- **Collision Damage Overlap Gizmo Duration** – How long collision gizmos remain visible.
- **Collision Damage Overlap Gizmo Color** – Color used for collision overlap gizmos.
- **Draw Structural Member Connection Gizmos** – Draws selected-member adjacency visualization.
- **Structural Member Connection Gizmo Color** – Color used for member-connection gizmos.

3. Prefab Workflow

3.1 Create a Structure Prefab

1. Build your structure in the scene and select its root object (the parent with the **StructuralGroup component**).
2. Drag the root object from the Hierarchy into the Project window to create a prefab asset.
3. DSB embeds the structure meshes into that prefab file automatically.

3.2 Editing Prefabbed Structures

Build modes are disabled while you are in Unity's **Prefab Mode** (Prefab Stage).

To edit a structure prefab correctly:

1. Exit Prefab Mode and return to the scene.
2. Select a **scene instance** of the prefab.
3. Make your build-mode edits on that scene instance.
4. In the Inspector, **Apply** the prefab overrides back to the prefab asset.

> **Known issue:** Undo for build-mode operations can become very slow (sometimes up to ~10 seconds per undo) when the prefab instance has a large number of diffs/overrides.
>
> **Workarounds:** - **Unpack** the prefab instance before large editing sessions, or - **Apply overrides frequently** to keep the override count low.

4. Runtime Systems

4.1 Applying Damage Through API

DSB automatically voxelizes members, surfaces, and connections at build time. These voxels become the destructible units used by the damage and debris systems, which are represented by a voxel object/component derived from **DSBVoxel** and implement **IDsbDamageable**.

Script / Type	Responsibilities
IDsbDamageable	Damage contract: `DamageResult ApplyDamage(float damage)`, plus health getters (`IsHealthAggregate`, `CurrentHealth`, `MaxHealth`).
DSBVoxel (abstract)	1. Stores an array of **DebrisData** (pre-cut meshes + materials). 2. Uses the runtime **Destruction Effects Manager** singleton to play pooled debris/effects, creating it automatically if needed. 3. On `Crumble()` spawns pooled debris, then destroys itself.

All voxels implement `IDsbDamageable`. Call `ApplyDamage(amount)` from your game scripts whenever a raycast, trigger, or explosion hits an object with an IDsbDamageable component attached. Damage accumulates per voxel and eventually leads to splitting or a call to `Crumble()`.

```
using Mayuns.DSB;

if (hit.collider.TryGetComponent<IDsbDamageable>(out var dmg))
    dmg.ApplyDamage(25f);
```

4.2 Applying Damage Through Collisions

Structures and detached structural groups can optionally take damage from rigidbody collision events. Collision damage can be toggled and adjusted in the global settings menu under **DSB window → Settings → Collision Damage**.

Collision settings reference:

- **Collision Damage Enabled**
  Master runtime toggle. When disabled, structures ignore collision-based damage (manual/API damage still works).
- **Collision Trigger Layers**
  Layer mask for the *incoming colliders* that are allowed to trigger collision damage events.
- **Collision Target Layers**
  Layer mask used when collecting nearby damageable voxels around the impact point. Only targets on these layers are considered.
- **Collision Impulse Threshold (N·s)** Minimum collision impulse required to start applying damage.
  - Higher values = fewer impacts qualify.

  - `0` means every valid collision can apply damage.

- **Collision Impulse Sensitivity**
  Multiplier that scales how quickly damage radius and base damage increase once impulse exceeds the threshold.

  - Higher values make heavy impacts ramp up more aggressively.

- **Collision Base Damage**
  Starting damage value before impulse scaling and distance falloff are applied.
- **Collision Base Radius (m)** Starting overlap radius around the contact point before impulse scaling is applied.
- **Collision Damage Max Radius (m)** Hard cap on the final overlap radius after scaling, preventing very large impacts from affecting unlimited area.
- Collision damage is calculated as follows:

  - A collision must come from an allowed **Collision Trigger Layer** and have an impulse greater than or equal to **Collision Impulse Threshold**.
  - `impulseOverThreshold = max(0, impulse - threshold)`
  - `impulseRatio = threshold > 0 ? impulseOverThreshold / threshold : impulseOverThreshold`
  - `impulseScalar = max(1, 1 + impulseRatio * sensitivity)`
  - `radius = clamp(baseRadius * impulseScalar, 0.001, maxRadius)`
  - `baseDamage = collisionBaseDamage * impulseScalar`
  - Final per-target damage uses a linear falloff from the contact point to the edge of the computed radius.

4.3 Navigation (Optional)

Navigation Overview

DSB ships with an optional helper that ties into Unity's AI Navigation package. Install `com.unity.ai.navigation` to keep pathfinding data aligned with collapsing structures.

StructureNavmeshRebaker

Attach this component to a **StructuralGroup component** to automatically refresh a `NavMeshSurface` whenever pieces crumble or large collapses occur.

- Auto-adds/configures a `NavMeshSurface` to collect child physics colliders, throttling rebakes via `debounceSeconds` and `minDelaySeconds`.
- Validates colliders on startup, warning about meshes that cannot be baked.

5. Extending DSB

5.1 Event System

DSB supports two approaches to customizing the audio-visual output of destruction:

- **Configure effects without code** – Assign an **Effects Profile** and optional **Material Overrides Profile** under **DSB window → Settings → Effects Profiles** to swap out particles/audio per event or per material. This is the fastest way to re-skin destruction; see Assigning Effects for authoring guidance.
- **Extend with code** – Subscribe to **DestructionSignals** to run your own logic in response to destruction events, optionally alongside inspector-driven effects. The remainder of this section focuses on this scripting-based workflow.

DestructionSignals Quick Reference

Event name	Parameters	Trigger
DestructionSignals.PieceCrumble	`(StructuralGroup group, Material material, Vector3 position)`	A structural piece (member, surface, or connection) crumbles.
DestructionSignals.MemberStress	`(StructuralGroup group, Material material, Vector3 position)`	The stress solver flags a beam as overstressed.
DestructionSignals.LargeCollapse	`(StructuralGroup group, Vector3 position)`	A detached group whose Rigidbody mass exceeds the configured **largeCollapseMassThreshold** collapses.
DestructionSignals.WindowShatter	`(StructuralGroup group, Material material, Vector3 position)`	A window panel is destroyed and shatters.
DestructionSignals.DebrisImpact	`(StructuralGroup group, Material material, Vector3 position)`	A flying debris collides with the world. Current raiser passes `group = null`, so ownership is not guaranteed.

Event Examples

Most code-driven integrations follow this pattern:

- Subscribe to **DestructionSignals** events to react to destruction moments (voxel crumbles, stress spikes, collapses, impacts). All events share the same signature (except LargeCollapse which isn't associated with a material), enabling structure-aware, material-aware, and location-based effects.
- Pair your scripts with inspector-driven content if desired. The **Effects Profile** (global defaults) and **Material Overrides Profile** (per-material overrides) assigned in **DSBGlobalSettings** continue to spawn the configured particles/audio while your code layers additional behavior on top.

5.2 Runtime Scripting API

```
using Mayuns.DSB;
using UnityEngine;
/// <summary>
/// Example of wiring custom logic into every stage of destruction.
/// </summary>
public class DsbEventBridge : MonoBehaviour
{
    void OnEnable()
    {
        // voxel-level crumble
        DestructionSignals.PieceCrumble += OnCrumble;
        // individual beam stress
        DestructionSignals.MemberStress += OnMemberStress;
        // wholesale collapse
        DestructionSignals.LargeCollapse += OnCollapse;
        // glass shatters
        DestructionSignals.WindowShatter += PlayGlassTinkle;
        // debris hits anything with a collider
        DestructionSignals.DebrisImpact += SpawnDustPuff;
    }
    void OnDisable()
    {
        DestructionSignals.PieceCrumble -= OnCrumble;
        DestructionSignals.MemberStress -= OnMemberStress;
        DestructionSignals.LargeCollapse -= OnCollapse;
        DestructionSignals.WindowShatter -= PlayGlassTinkle;
        DestructionSignals.DebrisImpact -= SpawnDustPuff;
    }

    /* ----------------- event handlers ----------------- */
    void OnCrumble(StructuralGroup group, Material m, Vector3 pos) {
        Debug.Log($"Chunk of {m.name} crumbled at {pos}");
    }
    void OnMemberStress(StructuralGroup group, Material m, Vector3 pos) {
        // e.g. flash a warning UI
    }
    void OnCollapse(StructuralGroup group, Material m, Vector3 pos) {
        // big boom - spawn camera shake, etc.
    }
    void PlayGlassTinkle(StructuralGroup group, Material m, Vector3 pos) {
        // trigger bonus shards
    }
    void SpawnDustPuff(StructuralGroup group, Material m, Vector3 pos) {
        // lighter particle on every debris impact
    }
}
```

Use the sample **DsbEventBridge** above as a template for custom integrations such as analytics hooks, dynamic lighting, or gameplay modifiers that respond to destruction events.

6. Troubleshooting Guide

Performance Issues

- **Stuttering / Low FPS during destruction events**
  - Increase **Fixed Timestep** in **Project Settings → Time**. A timestep of 0.02 is recommended for best performance.
  - Decrease **Max Active Debris** in **DSB window → Settings → Debris Behavior**. Keeping this under 3000 is recommended for best performance.
  - Decrease **Debris Lifetime** in **DSB window → Settings → Debris Behavior**. Keeping this under 30 seconds is recommended for best performance.
  - Assign a separate layer for debris in **DSB window → Settings → Layers → Debris Layer**, and modify the physics matrix in **Project Settings → Physics** to disable self-collisions or any unwanted collisions with other layers.

Editor & Build Tools

- **Build tools not working?**
  - Ensure gizmo visibility is enabled. (See Build Modes)
- **Undo in build modes is very slow on prefab instances (can take up to ~10 seconds)?**
  - This is a known issue when the instance has many prefab diffs/overrides.
  - Unpack the prefab instance before major edits, or apply overrides to the prefab asset regularly to reduce override count. (See Prefab Workflow)
- **Diagonal or orthogonal ghost beams don't appear.**
  - In **Overlay → Hide Axis** set **None** or **Orthogonal**.

Structure Management

- **Structure "vanished", meshes invisible, or missing pieces in a prefab?**
  - Exit Prefab Mode, edit the structure instance in the scene, use the **Regenerate Voxels** utility in the structure's inspector to regenerate missing geometry, then re-prefab it. Prefab overrides do not update DSB mesh references. (See Prefab Workflow)
- **Detached pieces look stretched or members and surfaces become distorted upon damage**
  - Non-uniform scaling is likely applied somewhere in the hierarchy. Use uniform scale on the structure and its parents.

Materials & Visuals

- **Surfaces are pink.**
  - Assign a material in **DSB window → Surface Build → Surface Material** or use **Apply Material** mode.
- **Stair steps rotate improperly.**
  - Step orientation depends on the local transform of the structure. Placed stairs will be oriented by whatever direction is "up" relative to the structure.

Runtime Effects

- **Want default effects without making your own library?**
  - Use the pre-made **Effects Library** under **Samples/** and assign it in **DSB window → Settings → Effects Profiles → Effects Profile**. (See Assigning Effects)
- **Pieces, debris, and detached groups disappear too quickly.**
  - Increase *Max Active Debris* or adjust debris & detached group lifetime in **DSB window → Settings → Debris Behavior**.
- **NavMesh doesn't rebake on collapse.**
  - Install `com.unity.ai.navigation` and add **StructureNavmeshRebaker** to your structure's root. (See Navigation)

7. Support & Legal

- **Email:** support@mayuns.com
- **Discord:** https://discord.gg/73GaMeP6JF
- **Website:** https://mayuns.com

Runtime & editor scripts are distributed under the standard **Unity Asset Store EULA**. All bundled textures, meshes, audio clips, particle presets, materials, and demo scenes were authored by Antonio Indindoli (dba Mayuns Technologies) and can be used in your shipped projects without any additional licensing restrictions or attribution.

Unity is a trademark or registered trademark of Unity Technologies or its affiliates in the U.S. and elsewhere. All other trademarks are the property of their respective owners.